

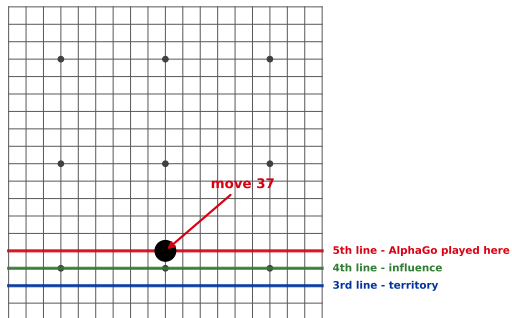
Reinforcement Learning

Part 1: the problem, and how to solve it when you know the rules

Part 1 of 7. Nothing learns in this deck – we define the problem and solve it exactly.

March 2016, Seoul. Move 37.

Schematic: opening play lives on the 3rd and 4th lines.
AlphaGo played the 5th.



AlphaGo, playing black against Lee Sedol, put a stone on the **fifth line**.

Centuries of theory put the opening on the third line (territory) or the fourth (influence). The fifth is *too far from the edge*.

AlphaGo's own policy network estimated the chance a **human** would play it at **1 in 10,000**.

Commentators called it a mistake. Lee Sedol, away from the board, returned and took **over 12 minutes** to answer. It won the game.

Nobody taught it that move

Everything so far

You are given (x, y) pairs.
Someone already knows the answer.

Here

You are given a *score*, sometimes,
long after you acted.

There was no database of “correct move 37s” to imitate. There was only: **play, and eventually find out whether you won.**

Be precise about the credit, though: AlphaGo learned from human games *and* self-play RL *and* tree search. Its successor **AlphaGo Zero** dropped the human games entirely and still surpassed it – that is the version that is purely reinforcement learning.

Where this chapter is going

Seven lectures. This one defines the problem; the rest attack it from different directions.

Lecture	The question it answers
1 The problem	What is an MDP, and how do I solve one exactly?
2 Learning from experience	What if nobody gives me the rules?
3 Deep value-based RL	What if there are too many states to write down?
4 Policy gradients	What if I optimise the behaviour directly?
5 Planning and self-play	What if I can think before I act?
6 RL for LLMs I	How does a language model become an agent?
7 RL for LLMs II	How do reasoning models actually get trained?

The last two are the payoff: the methods behind instruction-following chatbots and reasoning models are all the same equations you will meet in lectures 1–4, applied to text.

Outline

A different kind of problem

One decision: the bandit

Adding consequences: the MDP

Value

Solving it exactly

Why nothing so far applies

There is no correct answer to imitate. Only consequences.

Reward is not a loss function

This is the frame that explains why none of chapters 1–10 apply directly.

Three properties a loss does not have

1. **Delayed.** The move that lost you the game was probably not the last one. Which of the 200 moves gets the blame? (*credit assignment*)
2. **Sparse.** Most steps score zero. A robot that has never once reached the goal has no signal at all to improve from.
3. **Not differentiable.** “The user preferred this answer.” “The code passed the tests.” There is no $\partial \text{reward} / \partial \theta$ to descend.

And one property it *does* have that is worse: **the data depends on the policy.** Change how the agent behaves and you change the distribution it learns from. The i.i.d. assumption underneath every previous chapter is gone.

Two kinds of task

Episodic

There is an end. A game finishes, a robot reaches the goal, an answer is generated.

The run from start to end is an **episode**, and you can talk about the total reward it collected.

Chess, tic-tac-toe, one chatbot reply.

Continuing

There is no end. The process just keeps going.

“Total reward” is now an infinite sum, so it needs taming before it means anything.

A data-centre cooling controller, a trading system, a recommender.

This distinction is the reason the discount factor γ exists, which we get to shortly. For now: almost everything in this chapter is episodic, and LLM training very much is – the episode is one response.

Start with one decision

Delete the state, delete the future. What is left is still hard.

Ten ads and one slot

You run a website. You have **10 banner ads** and one slot. Each visitor sees one ad and either clicks or does not.

The k -armed bandit

- ▶ k actions, each with an unknown mean reward μ_a .
- ▶ Pick one, observe a noisy reward, repeat T times.
- ▶ **No state.** Nothing you do changes the world.
- ▶ **No delay.** The reward arrives immediately.

Two of the three hard things about RL are gone. Credit assignment: gone, there is one action. Delayed reward: gone.

What remains is the third: **you only learn about the action you actually took.**

That single property is enough to make the problem non-trivial, and it is the cleanest place to meet it.

Predict first

Here is the obvious plan. It is what most people propose, and it is what a naive A/B test does.

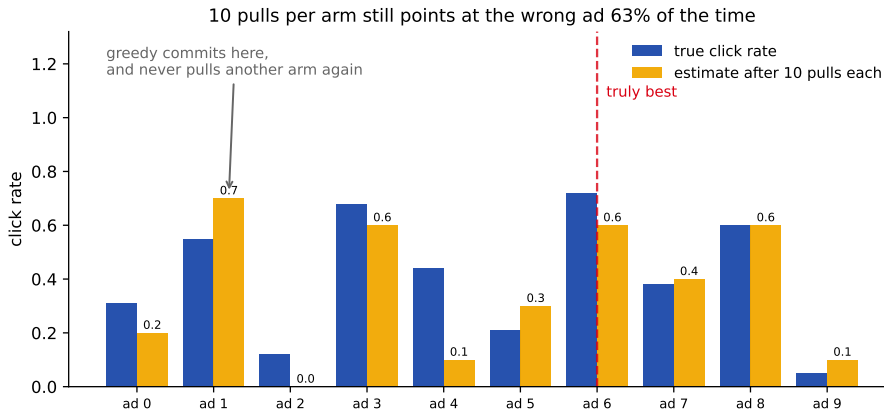
The obvious plan

1. Show each of the 10 ads to 10 visitors each. (100 visitors spent learning.)
2. Whichever had the most clicks, show it to everyone from then on.

How often does this commit to the **wrong** ad?

Guess a number before the next slide. The true click rates range from 0.05 to 0.72, and the best two are 0.72 and 0.68.

Two thirds of the time



Measured over 20,000 replications: after 10 pulls each, the empirically best ad is **not** the truly best ad **63.3%** of the time. In the draw shown, ad 1 (true rate 0.55) got lucky and beat ad 6 (true rate 0.72).

And because the plan then stops exploring, it **never finds out**. The mistake is permanent, 12 / 39

Regret: the thing you are actually minimising

$$\text{Regret}(T) = \underbrace{T\mu^*}_{\text{if you had always played the best arm}} - \underbrace{\sum_{t=1}^T \mu_{a_t}}_{\text{what you actually got}}$$

You cannot measure this while running – it needs μ^* , which is exactly what you do not know. It is an *analysis* tool, not a training signal. But it makes the goal precise:

Linear regret $\Theta(T)$ – you keep making the same mistake at a constant rate. Never stops costing you.

Logarithmic regret $O(\log T)$ – the mistakes get rarer over time. This is the best achievable (Lai & Robbins, 1985).

“Explore forever” and “never explore” are both linear. The art is in between.

Three ways to decide what to try

ϵ -greedy – explore at random, sometimes

With probability $1 - \epsilon$ take $\arg \max_a \hat{\mu}_a$; otherwise pick uniformly at random. *Dumb but effective*. Its flaw: it explores the obviously terrible arms just as often as the promising ones, forever.

Upper Confidence Bound (UCB) – optimism under uncertainty

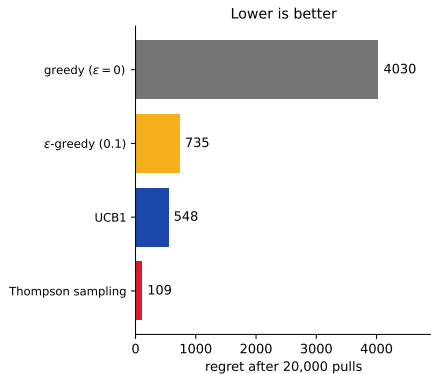
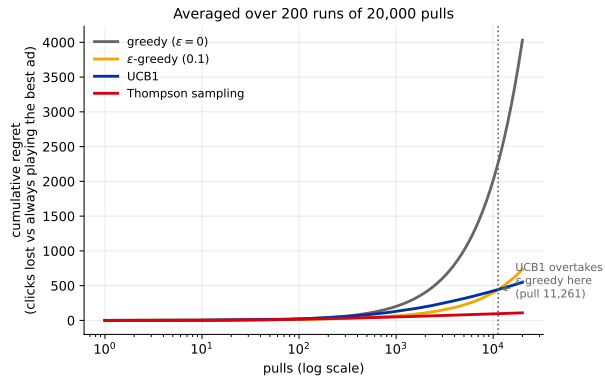
$$a_t = \arg \max_a \left[\hat{\mu}_a + \sqrt{\frac{2 \ln t}{N_a}} \right]$$

Take the best *plausible* value, not the best estimate. The bonus shrinks as N_a grows, so an arm you have tried a lot stops looking artificially good.

Thompson sampling – act on a sampled belief

Keep a posterior over each μ_a (here $\text{Beta}(\alpha_a, \beta_a)$). *Sample* one value from each, play the argmax, update. Older than the others (Thompson, 1933) and usually the best.

What each one costs



200 runs, 20,000 pulls, 10 ads. Greedy ($\epsilon = 0$) is the flat-out disaster: regret **4030**, and it spends only **18.5%** of its pulls on the best ad. Thompson sampling ends at **109** and spends **94.1%**.

The uncomfortable detail in that plot

UCB has the better guarantee and still loses – for a while

UCB1 has $O(\log T)$ regret. ϵ -greedy with fixed ϵ has *linear* regret. Theory says UCB must win eventually.

Measured on this problem: ϵ -greedy is **ahead until pull 11,261**.

	regret @ 1,500	regret @ 20,000
ϵ -greedy (0.1)	86	735
UCB1	166	548

The lesson, and it applies to everything later in this chapter: an asymptotic guarantee tells you who wins at infinity. If your campaign is 2,000 impressions long, the algorithm with the worse guarantee is the right choice. Always ask what horizon a result is quoted at.

Where the bandit *is* the whole job

This is not a toy chapter you pass through on the way to the real thing. Enormous amounts of deployed machine learning are bandits:

- ▶ **Which headline / thumbnail / layout.** An A/B test that stops early and commits is the naive plan from three slides ago. A bandit keeps earning while it learns.
- ▶ **Recommendations and ads.** Which item to show, out of millions.
- ▶ **Clinical trials.** The original motivation – allocating patients to treatments while people are still being treated.
- ▶ **Hyperparameter search.** Successive halving and Hyperband are bandit algorithms.

Contextual bandits: one step closer

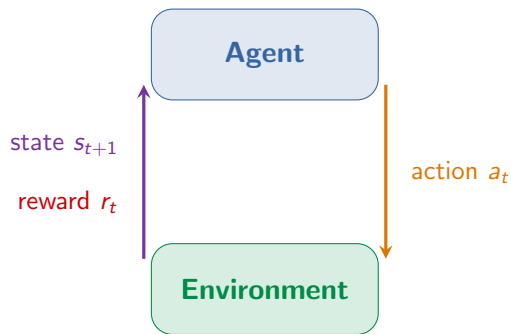
Now you see a **context** first (this user, this time of day) and pick an action for it. The policy becomes $\pi(a \mid x)$ – a function you can fit with any model from chapters 1–10.

What is still missing: your action does not change the next context. No consequences, no future to plan for. Add that, and you have the full problem.

Agent, environment, reward

Now what you do changes what you face next.

The loop



The agent's whole job:

find a **policy** $\pi(a|s)$ – how likely it is to take action a when in state s – that makes

the *total* reward

$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$
as large as possible.

Note what the agent does *not* get: no example of a good action, no gradient of “goodness” with respect to what it did. Only the consequence.

The MDP: five symbols for the whole setting

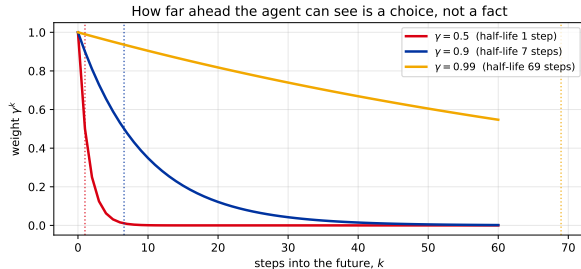
A **Markov Decision Process (MDP)** is the standard way to write all of this down:

$$\text{MDP} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

Symbol	Name	In the gridworld we are about to use
$\mathcal{S}$	states	which square you are standing on
$\mathcal{A}$	actions	up, down, left, right
$P(s' s, a)$	transition	where you actually end up (you may slip)
$R(s, a)$	reward	-0.04 per move, $+1$ at the goal, -1 in the pit
γ	discount	how much a future reward is worth now

The Markov property is the assumption hiding in the name: the future depends on the *current* state only, not on how you got there. It is what makes any of this tractable, and it is frequently a lie in practice.

Why discount at all?



γ answers one question: *how much is a reward worth if it arrives later?*

- ▶ γ near 0 – greedy, near-sighted. Only the next few steps matter.
- ▶ γ near 1 – patient. Willing to suffer now for a payoff much later.

It is not only a modelling taste: $\gamma < 1$ also keeps the infinite sum finite, which is what makes the whole thing well defined.

The discount is not a detail – it changes the plan

Same world, same rewards - the discount decides which one is worth walking to
(the decision flips at $\gamma \approx 0.68$)



+1 one step left, +10 seven steps right. Only γ differs between the rows.

Check it yourself. With $G_t = r_t + \gamma r_{t+1} + \dots$ the first reward you collect is *undiscounted*, so a reward N steps away carries γ^{N-1} – at $\gamma = 0.5$ that is $0.5^0 \times 1 = 1.000$ going left against $0.5^6 \times 10 = 0.156$ going right. The agent walks **away** from the bigger prize. Equating the two gives the flip at $\gamma = (1/10)^{1/6} \approx 0.68$.

A policy is not always a lookup table

Deterministic $\pi(s) = a$

One action per state. “In this square, go left.”

For an MDP with known dynamics, there is *a/ways* an optimal policy of this kind. You never need randomness to play a gridworld well.

Stochastic $\pi(a | s)$

A distribution over actions. “70% left, 30% up.”

Needed when the state is only partially observed, when an opponent can exploit you, and – practically – because it is what makes a policy differentiable.

Worth flagging now: a language model *is* a stochastic policy. Sampling the next token from a softmax is exactly $a \sim \pi(\cdot | s)$. Lecture 6 does nothing more exotic than take that sentence literally.

How good is this position?

If you can answer that, the policy follows for free.

Two value functions

State value $V^\pi(s)$

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_k \gamma^k r_{t+k} \mid s_t = s \right]$$

“Starting here and behaving like π , what do I collect?”

Action value $Q^\pi(s, a)$

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_k \gamma^k r_{t+k} \mid s_t = s, a_t = a \right]$$

“...if I take a first, and behave like π afterwards?”

Q is the more useful of the two, and the reason is practical: to *act* from V you need to know where each action leads. To act from Q you just compare numbers you already have. That single difference is why most algorithms in this chapter learn Q .

The Bellman equation, derived

Everything rests on one observation: **the return is recursive**. $G_t = r_t + \gamma G_{t+1}$. Take expectations of that and turn the handle.

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$$

definition

$$= \mathbb{E}_\pi[r_t + \gamma G_{t+1} \mid s_t = s]$$

$$G_t = r_t + \gamma G_{t+1}$$

$$= \sum_a \pi(a|s) \left[R(s, a) + \gamma \mathbb{E}_\pi[G_{t+1} \mid s, a] \right]$$

split on the action

$$= \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) \mathbb{E}_\pi[G_{t+1} \mid s'] \right]$$

split on where you land

$$= \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right]$$

Markov

Read it aloud: *what you get immediately, plus the discounted value of wherever you end up.*
The last step is where Markov earns its keep – without it, $\mathbb{E}[G_{t+1}]$ would depend on the whole history, not just on s' .

Optimality puts a max where the average was

The equation above evaluates a policy. To find the *best* one, stop averaging over what π would do and take the best action instead:

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right]$$
$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$$

Expectation equation: linear

Fix π and it is $|S|$ linear equations in $|S|$ unknowns. You could solve it with a matrix inverse.

Optimality equation: not

The max makes it nonlinear. No matrix inverse, no closed form. **This is why every method from here on iterates.**

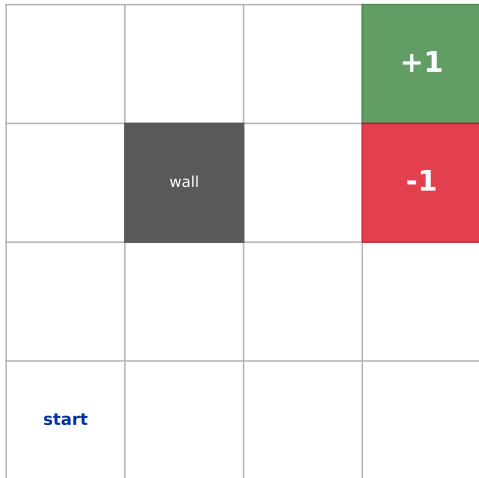
Everything that follows – value iteration, Q-learning, DQN, the critic inside PPO – is a way of **solving or approximating this one equation.**

When you know the rules

Dynamic programming: the answer, computed rather than learned.

Our running example

Every move costs -0.04 . Reach $+1$, avoid -1 .
Actions slip sideways with probability 0.1 each.



A 4×4 world. Start bottom-left, $+1$ top-right, -1 just below it, a wall in the middle.

Two details that make it interesting:

- ▶ Every move costs -0.04 , so dawdling is punished. Without this the agent would be indifferent to taking the scenic route.
- ▶ Actions are **unreliable**: you go where you intended with probability 0.8 , and slip 90° to either side with probability 0.1 each.

That second one is why walking right past the -1 is a bad idea even though it is the shortest path.

Step 1: evaluate a policy

Given a fixed policy π , how good is it? Turn the Bellman *expectation* equation into an assignment and sweep it over every state, repeatedly:

$$V_{k+1}(s) \leftarrow \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s') \right]$$

Why this converges

The update is a **contraction**: applying it to two different value functions brings them closer together by a factor of γ . So the distance to the true V^π shrinks like γ^k – geometrically, from any starting point, with a unique fixed point.

At $\gamma = 0.9$, that is a factor of 10 every 22 sweeps. Slow, but guaranteed.

Start V_0 at anything (zeros are fine). Sweep until nothing moves.

Step 2: improve it

Now that you know what π is worth, act greedily with respect to those values:

$$\pi'(s) = \arg \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right]$$

The policy improvement theorem

$V^{\pi'}(s) \geq V^\pi(s)$ for **every** state s . Not on average – everywhere. And if $\pi' = \pi$, then π already satisfies the Bellman optimality equation, so it is optimal.

This is a stronger promise than anything in supervised learning, where a gradient step can make some examples worse. Here, greedy improvement never makes any state worse.

Since there are finitely many deterministic policies and each round strictly improves, this **must** terminate at the optimum.

Two famous algorithms, one knob

Alternate the two steps. The only question is **how long you evaluate before you re-greedy**:

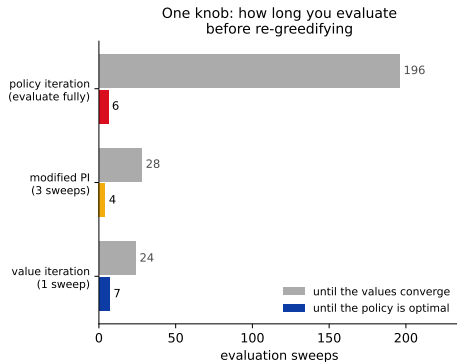
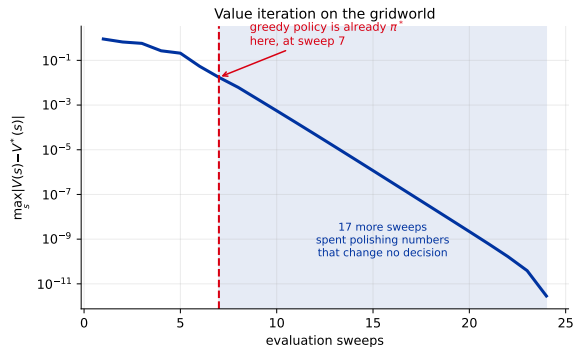
Evaluation sweeps per round	Name	Character
Until convergence	Policy iteration	Few rounds, each expensive
Exactly 1	Value iteration	Many rounds, each cheap
Some k in between	Modified policy iteration	Usually the fastest in practice

With one sweep the two steps collapse into a single line, which is the form you will actually see written down:

$$V_{k+1}(s) \leftarrow \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V_k(s') \right]$$

That is **value iteration**, and it is just the Bellman optimality equation used as an assignment.

Measured on our gridworld

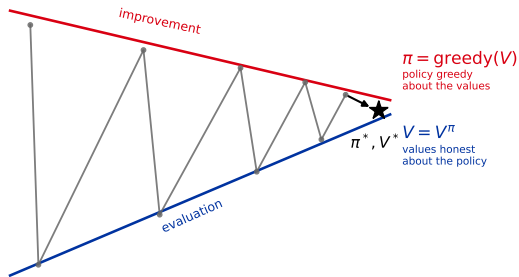


The result worth remembering: value iteration has the optimal *policy* at sweep **7**, then spends **17 more sweeps** refining values that change no decision. Full policy iteration is worse: it finds π^* at sweep 6 and then grinds to sweep **196** evaluating policies to machine precision on its way there.

Converged values are *sufficient* for a good policy, never *necessary*.

Generalised policy iteration

Generalised policy iteration



Every method in this chapter bounces between these two lines. They differ only in how far they go each time.

Two forces, pulling in different directions:

- **Evaluation** drags V towards the truth about the current π , which makes π look non-greedy.
- **Improvement** makes π greedy for the current V , which makes V wrong again.

They only stop fighting at π^*, V^* .

Keep this picture. Q-learning, DQN, actor-critic, PPO and GRPO are all this diagram with different ways of doing the two steps approximately, from sam-

Predict first

We are about to solve the gridworld exactly. Before the numbers appear:

Look at the square **directly below the -1 pit**.

The goal $+1$ sits two squares straight up, just past the pit. Does the optimal policy there point **up** – the short way, squeezing past the pit – or does it point **away**?

Commit to an answer, and to a reason. Remember that actions slip sideways one time in five.

Solved, exactly

$V^*(s)$ after value iteration
(green = worth being here, red = not)

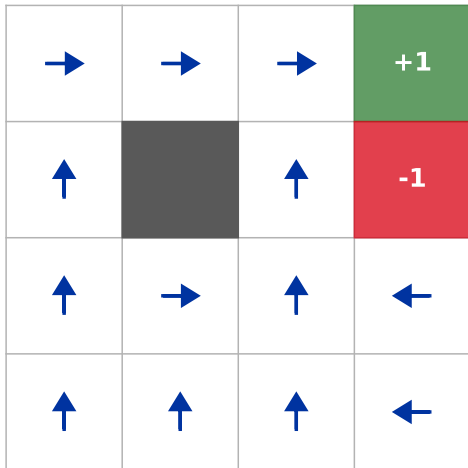
0.51	0.65	0.80	1.00
0.40		0.49	-1.00
0.30	0.25	0.34	0.13
0.21	0.18	0.24	0.16

Value iteration: apply the Bellman update to every square, repeatedly, until nothing changes. Converged here in **23 sweeps**.

- ▶ $V^*(\text{start}) = 0.2075$ – positive, but modest: the goal is far and every step costs.
- ▶ The square beside the goal is worth 0.7954.
- ▶ Notice the value *gradient* pointing away from the pit. Squares near it are worth less even when they are close to the goal.

And the policy falls out of it

The greedy policy $\pi^*(s) = \arg \max_a Q^*(s, a)$
read straight off the values



$\pi^*(s) = \arg \max_a Q^*(s, a)$ – once you know the values, choosing is trivial.

The answer to the prediction: the square below the pit points **left**, not up.

Going up means walking alongside the pit, and a 10% slip lands you in it and ends the episode at -1 . The long way round costs a few more -0.04 steps and is *still* worth more.

The agent is not avoiding the pit. It is avoiding *the chance* of the pit.

What we just did was cheating

Model-based

You know $P(s'|s, a)$ and R .
Plan by computing.

Value iteration, AlphaGo's search

Model-free

You know nothing.
Learn by trying.

Q-learning, DQN, PPO

A robot does not come with a probability table for how its wheels slip on this particular carpet. A chatbot has no model of how a human will react. **Model-free is the normal case.**

Every number on the last three slides came from P . Take P away and none of it can be computed – it has to be **estimated from experience**. That is the next lecture.

Recap

- ▶ RL is not supervised learning with a harder loss. The signal is **delayed, sparse, non-differentiable**, and the data depends on the policy that collected it.
- ▶ The **bandit** is exploration with everything else removed – and it is deployed technology, not a warm-up. Committing after a short exploration phase picks the wrong arm **63.3%** of the time here.
- ▶ Asymptotic guarantees are about infinity. UCB1 beats ϵ -greedy on this problem only after **pull 11,261**.
- ▶ An **MDP** adds states and consequences. γ decides how far ahead the agent looks, and can flip the optimal policy.
- ▶ The **Bellman equation** is $G_t = r_t + \gamma G_{t+1}$ with expectations taken. The optimality version has a max, so it must be solved by iteration.
- ▶ **Generalised policy iteration** – evaluate, then be greedy, repeat – is the skeleton of every algorithm in the remaining six lectures.

Next: the agent loses the map. No P , no R – only samples. Monte Carlo, temporal difference, SARSA and Q-learning.